arXiv:2502.13298v2 [cs.CL] 26 Dec 2025

Improving Multi-turn Task Completion in Task-Oriented Dialog Systems via Prompt Chaining and Fine-Grained Feedback

Moghis Fereidouni[1], **Md Sajid Ahmed**[2], **Adib Mosharrof**[1], **A. B. Siddique**[1]

[1]University of Kentucky, [2]Independent Researcher

{moghis.fereidouni, adib.mosharrof, ab.siddique}@uky.edu, sajid.ahmed1@northsouth.edu

Abstract

Task-oriented dialog (TOD) systems facilitate users in accomplishing complex, multi-turn tasks through natural language. While instruction-tuned large language models (LLMs) have demonstrated strong performance on a range of single-turn NLP tasks, they often struggle with reliable multi-turn task completion in TOD settings, particularly when generating API calls required to interact with external systems. To address this, we introduce RealTOD, a novel framework that improves LLM-based TOD systems through (*1*) prompt chaining and (*2*) fine-grained feedback. Prompt chaining enables zero-shot generalization to new domains by automatically synthesizing a schema-aligned in-context example for the target task. Fine-grained feedback verifies each generated API call against the domain schema, identifies specific errors, and provides targeted correction prompts. To evaluate task completion reliability, we introduce full API Call Accuracy as a robust metric, along with detailed sub-metrics to capture common failure modes. We conduct extensive experiments on the SGD and BiTOD benchmarks using four LLMs. RealTOD improves Full API accuracy, surpassing state-of-the-art AutoTOD by 37.10% on SGD and supervised learning-based baseline SimpleTOD by 10.32% on BiTOD. Human evaluations further confirm that LLMs integrated with RealTOD achieve superior task completion, fluency, and informativeness compared to existing methods.

Introduction

Task-oriented dialog (TOD) systems enable users to accomplish tasks, such as booking flights, making restaurant reservations, and managing appointments, through multi-turn, natural language interactions (He et al. 2022). To successfully complete tasks, these systems must understand user intent, retrieve task-specific information from both user and external systems, and generate coherent responses to guide the user toward task completion (e.g., successful reservation). However, traditional TOD systems require extensive domain-specific fine-tuning using manually annotated datasets (Xu et al. 2024; Mi, Wang, and Li 2022), making them laborious and expensive to scale across domains.

Recent advances in large language models (LLMs) (Radford et al. 2019; Brown et al. 2020; Chung et al. 2024; Shu et al. 2024) have significantly improved performance across a wide range of single-turn natural language processing (NLP) tasks, such as text classification (Sun et al. 2023; Wang, Pang, and Lin 2023; Zhang et al. 2024b), summarization (Pu, Gao, and Wan 2023; Zhang et al. 2024a; Van Veen et al. 2023), and machine translation (Cui et al. 2025; Wang et al. 2023; Gain, Bandyopadhyay, and Ekbal 2025). In addition to these single-step tasks, LLMs have also shown strong capabilities in multi-turn, open-domain dialog settings (Thoppilan et al. 2022; Naveed et al. 2023; Dubey et al. 2024; Achiam et al. 2023), where coherent response generation given dialog context is critical.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

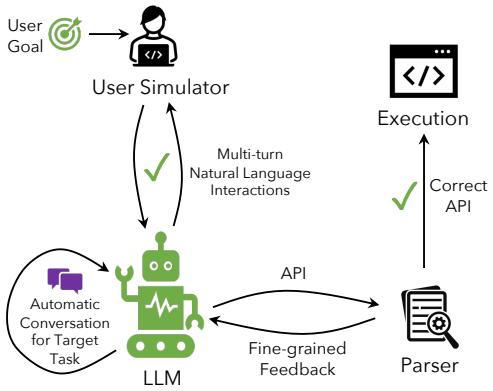

Figure 1: Overview of RealTOD: Prompt chaining circumvents the need for human-curated dialog for each task, and fine-grained feedback loop from the API parser significantly improves task completion rates.

Motivated by these advances, recent work has explored leveraging off-the-shelf LLMs for zero-shot TOD settings (Xu et al. 2024). These models can generate fluent and contextually appropriate responses, and with careful prompting, can even be controlled to follow task schemas. However, despite their promise, LLMs often struggle with reliable task completion, particularly when generating API calls required to interact with external systems. Common issues include hallucinated API results, incorrect method names, missing required parameters, and invalid slot-value pairs (Shinn et al. 2023; Jain et al. 2024; Song et al. 2025), all of which can lead to execution failures and incomplete tasks. These issues reveal a critical gap between LLMs' natural language fluency and their ability to execute tasks reliably in multi-turn dialogs, presenting a major barrier to their practical deployment.

To address these issues, we propose RealTOD, a novel framework that enhances LLM-based TOD systems through

(*i*) prompt chaining and (*ii*) fine-grained feedback. LLMs' performance on single-turn NLP tasks has been shown to improve significantly when provided with in-context examples (Liu et al. 2022b; Dong et al. 2024; Agarwal et al. 2024). However, applying this strategy in TOD settings presents a practical challenge: acquiring multi-turn dialogs that are both realistic and schema-compliant for each target domain or task is labor-intensive and does not scale. RealTOD addresses this challenge through prompt chaining, which automatically transforms dialogs from a source domain into in-context demonstrations that align with the schema of an unseen target domain. This approach substantially improves zero-shot generalization to unseen tasks in unseen domains without requiring hand-crafted example dialogs for each task. In TOD systems, successful task completion often hinges on generating a correct API call that invokes the correct method and includes all required parameters with valid slot-value pairs. Even small errors in API calls can result in execution failures, ultimately preventing task completion. To mitigate this, RealTOD incorporates a fine-grained feedback mechanism that verifies each generated API call against the target domain specification, identifies specific errors, and provides targeted correction prompts. This iterative process significantly improves the reliability of API calls and, in turn, the overall task completion rate.

In addition to improving task completion in TOD systems, we introduce a rigorous evaluation metric for measuring task success. Existing metrics focus on n-gram-based similarity metrics (Papineni et al. 2002; Lin 2004), which do not reflect whether a task has been successfully executed. To better align evaluation with practical deployment needs, we adopt full API Call Accuracy as our primary metric. An API call is counted as correct only if the method name, parameter names, associated parameter values, and operator match the reference. We further decompose this metric into fine-grained sub-metrics, Method Name Accuracy, Parameter Name Accuracy, Parameter Value Accuracy, and Operator Accuracy, to diagnose specific failure modes. This evaluation setup captures end-to-end task completion reliability. On the other hand, we evaluate the quality of natural language responses using BERTScore (Zhang et al. 2019), which assesses semantic similarity between generated and reference responses via contextual embeddings. We also measure diversity (Nekvinda and Dušek 2021) of the generated text, providing a more robust evaluation than traditional n-gram-based metrics.

We evaluate RealTOD on two benchmark datasets, SGD (Rastogi et al. 2020) and BiTOD (Lin et al. 2021b), using four LLMs: two proprietary models (GPT-4o and Claude) and two open-source models (DeepSeek and LLaMA). Our results show that RealTOD significantly improves API accuracy across all models and datasets. On SGD, it surpasses AutoTOD (Xu et al. 2024) by 37.10% in full API accuracy, while on BiTOD, it outperforms supervised fine-tuned SimpleTOD (Hosseini-Asl et al. 2020) by 10.32%. Human evaluations confirm that LLMs integrated with RealTOD generate more fluent, informative, and effective task completions than baseline models. Our ablation study demonstrates that both prompt chaining and fine-grained feedback contribute to improved multi-turn dialog quality and reliable task completion.

Related Works

Fine-Tuned Task-Oriented Dialog Systems. TOD systems are typically classified into pipeline-based and end-to-end approaches. Pipeline-based methods (Williams and Young 2007; Lee 2013; Lee et al. 2009; Peng et al. 2020; Chen et al. 2019) decompose the system into modular components—natural language understanding, dialog state tracking, policy learning, and natural language generation—allowing independent optimization of each module. In contrast, end-to-end approaches (Hosseini-Asl et al. 2020; Madotto, Wu, and Fung 2018; Su et al. 2022; Mosharrof, Maqbool, and Siddique 2023; Siddique et al. 2022; Lei et al. 2018; Lin et al. 2020; Imrattanatrai and Fukuda 2023) generate responses directly, bypassing these modules. A major drawback of these fine-tuned methods is their reliance on high-quality labeled data, which can be a significant limitation.

LLM-Powered Systems. The rise of LLMs has led to the development of various intelligent systems, which can be broadly categorized into three classes. The first class includes Web Agents, which facilitate online interactions for information retrieval and task execution (Yao et al. 2023; Kim et al. 2024; Ma et al. 2023; Fereidouni, Mosharrof, and Siddique 2024; Yao et al. 2022; Sridhar et al. 2023; Furuta et al. 2024). The second class consists of Mobile Agents, which focus on optimizing LLM-based decision-making for performing diverse tasks on mobile applications (Bai et al. 2024; Lee et al. 2023; Wen et al. 2024, 2023; Wang et al. 2024b,a). The third and most relevant class to our work is LLM-powered TOD Systems (Chung et al. 2023; Mi, Wang, and Li 2022; Gao et al. 2023; Hudeček and Dusek 2023; Labruna et al. 2023). Specifically, AutoTOD (Xu et al. 2024) shares similarities with our approach; however, AutoTOD does not account for the possibility of LLMs making errors in generating API calls and lacks proper evaluation of API accuracy.

User Simulators. One of the earliest data-driven user simulators is (Eckert, Levin, and Pieraccini 1997), where user actions are generated probabilistically based on system actions. Additionally, there have been many advancements in data-driven user simulation. For instance, transformer-based architectures have been leveraged for domain-independent simulation (Lin et al. 2021a, 2022) with GPT-based models integrating goal state tracking (Liu et al. 2022a). Reinforcement learning has also been applied to fine-tune generative simulators (Tseng et al. 2021; Cheng et al. 2022). More recently, in-context learning with LLMs has enabled user simulation without fine-tuning, (Terragni et al. 2023; Davidson et al. 2023). Similar to (Lin et al. 2021a, 2022; Liu et al. 2022a), our user simulator employs transformer architectures.

Proposed Framework: RealTOD

We introduce RealTOD, a task-completion-centric, TOD framework that eliminates the need for fine-tuning while seamlessly scaling to new domains. While recent LLM-based TOD systems can generate fluent and contextually appropriate responses, they often fail to execute tasks reliably, particularly when producing API calls needed to interact with external systems. To address this, RealTOD introduces two key innovations: (*1*) prompt chaining, which first transforms a source-domain dialog into a schema-aligned demonstration

for a new target task, and then uses the generated dialog as an in-context example, eliminating the need for hand-crafted demonstrations, to enable zero-shot generalization to the target domain on-the-fly; and (*2*) fine-grained feedback, which verifies each generated API call against the domain schema, identifies specific errors, and provides targeted correction prompts to improve task execution reliability.

Problem Formulation

We formulate multi-turn task completion as a conditional sequence generation problem, where the LLM produces natural language responses or API calls to help users achieve their goals across relevant domains. Each API call includes the method name, a dictionary of parameter names, and their corresponding values.

Formally, a domain $d_x \in D$ is characterized by a domain schema, which consists of a set of user intents $\mathcal{I}_{d_x}$. An intent represents a specific goal the user aims to achieve through the multi-turn conversation. For example, in the "Flights" domain, an intent might be "Book a Flight". Each intent $i \in \mathcal{I}_{d_x}$ is associated with a set of slots $\mathcal{S}_i$, where each slot s captures relevant constraints to fulfilling the intent. For example, the intent of "Book a Flight" may involve slots such as "departure city" and "destination city". We define a slot s as a tuple:

$$s = (\texttt{name}(s), \texttt{is_required}(s), \texttt{values}(s))$$

where $\texttt{name}(.)$ specifies the slot's name (e.g., "departure city"), $\texttt{is_required}(.)$ is a boolean flag indicating whether the slot is mandatory, and $\texttt{values}(.)$ specifies a predefined set of possible values for categorical slots (e.g., "business class", "economy class" in the "Flights" domain). If the slot accepts free-form inputs, this field remains empty. For brevity, we will refer to the name of a slot $\texttt{name}(s)$ as s_m. Formally, the schema for a domain d_x is represented as:

$$\Sigma_{d_x} = (d_x, \mathcal{I}_{d_x}, \{\mathcal{S}_i \mid i \in \mathcal{I}_{d_x}\}).$$

In addition to generating natural language responses, the model may need to retrieve information from external systems or execute actions via API calls to accurately fulfill a user's goal. Each API call corresponds to a specific intent in a domain and a set of specified constraints, represented as slot-value pairs. Formally, an API call a_n is defined as: $a_n = \texttt{API}(\texttt{method} = i, \texttt{parameters} = \{(s_m, v), (\cdots) \mid s_m \in \mathcal{S}_i\})$, where i is the intent, s_m is the slot name, and v is its assigned value. For instance, in the "Flights" domain, an API call for booking a flight may look like: $\texttt{API}(\texttt{method} =$ "Book_a_Flight", $\texttt{parameters} = \{($"departure_city", "New York"$)$, ("destination", "London"$), (\cdots)\})$.

A dialog session in a domain d_x consists of a sequence of user utterances and system responses across multiple turns. We define a session $\mathcal{T}_{d_x}$ of up to T turns as:

$$\mathcal{T}_{d_x} = \big((u_1, r_1), (u_2, r_2), \ldots, (u_T, r_T)\big)$$

where u_t is the user's utterance at turn t, r_t is the system's response at turn t, which can either be a natural language reply or an API call. The dialog history up to turn t, denoted as H_t, consists of all previous exchanges up to and including the current user utterance: $H_t = \{(u_1, r_1), (u_2, r_2), \ldots, (u_{t-1}, r_{t-1}), u_t\}$.

Prompt Chaining

In-context examples have been shown to substantially improve LLM performance in a wide range of single-turn NLP tasks (Liu et al. 2022b; Dong et al. 2024; Agarwal et al. 2024). However, their potential in multi-turn, schema-constrained dialog tasks, such as those in TOD systems, remains underexplored. Unlike single-step tasks, TOD requires models to track dialog history, collect slot values across multiple turns, and conform to structured API schemas. Manually creating such in-context examples for every target domain is labor-intensive and hinders scalability. To address this, RealTOD introduces a two-stage prompt chaining mechanism that enables zero-shot generalization to new domains without requiring any hand-crafted dialogs. Specifically, RealTOD employs a two-stage prompt chaining mechanism, which consists of two sequential prompting phases: (*i*) example dialog generation that transforms an example dialog from a source domain into a target domain while maintaining task-specific consistency; and (*ii*) task adaptation that leverages the generated example dialog for in-context learning in the target domain.

Example Dialog Generation. The first phase constructs an example dialog in the target domain by leveraging the schema mapping between the source domain and target domain. Formally, the inputs to LLM in this phase include the source domain schema Σ_{d_x}, an example dialog $\mathcal{T}_{d_x}$ in the source domain, an instruction prompt P_1 specifying the transformation process, and the target domain schema Σ_{d_y}. The output is a new example dialog $\mathcal{T}_{d_y}$ that aligns with the intents $\mathcal{I}_{d_x}$ and associated slots $\mathcal{S}_{i_x}$ in the target domain d_y.

Task Adaptation. Once the example dialog $\mathcal{T}_{d_y}$ in target domain d_y is generated, the second phase leverages this as an in-context learning example to enhance the model's adaptation in the target domain. At each dialog turn t, the inputs to the LLM include the target domain schema Σ_{d_y}, the generated example dialog $\mathcal{T}_{d_y}$, the dialog history up to turn t (denoted as H_t), and an instruction prompt P_2 that guides the response generation process.

The LLM then produces the system response r_t, which can be either a natural language reply or an API call, depending on the current task context. Since a single dialog may span multiple domains, we can denote the set of target domains involved in a dialog session as $\{d_1, d_2, \ldots, d_m\} \subseteq D$, and extend to the formulation to condition on all relevant domain schemas $\{\Sigma_{d_j}\}_{j=1}^{m}$.

Instruction Prompts. The prompt P_1 begins with a task description on generating a dialog from a schema, then presents domain_X's schema and its sample conversation. It instructs the LLM to analyze this structure, apply it to domain_Y, and generate a corresponding conversation. (For the full prompt P_1, see Appendix D.) The instruction prompt P_2 consists of two main parts: a task description and general guidelines. It directs the system to collect required slot values before API calls and use search results for accurate responses. The guidelines emphasize limiting slot requests per turn and confirming user inputs before invoking the API call. (For the full prompt P_2, see Appendix D.)

Fine-Grained Feedback

In TOD systems, successful task completion often depends on generating precise API calls that invoke the correct method and include all required parameters with well-formed slot-value pairs. However, LLMs, despite their fluency, operate in a free-form text generation mode and must reason over a long dialog history, while adhering to rigid schema constraints. This often results in API calls that are incomplete, inconsistent, or even invalid, issues that cannot be reliably prevented through prompting alone. To minimize these errors and ensure successful task execution, RealTOD integrates a fine-grained feedback mechanism via a generic API parser. Given a domain schema Σ_{d_x} and an API call a, the parser verifies the correctness of the request before execution. If the API call conforms to the schema, it is passed for execution; otherwise, the parser provides fine-grained feedback to the LLM for correction. The verification process identifies three types of errors: (*i*) incorrect method name, where the API method does not match any intent $i \notin \mathcal{I}_d$; (*ii*) incorrect slot name, where a provided slot is not defined in the schema $s_m \notin \mathcal{S}_i$ for the given intent; and (*iii*) missing required slots, where required slots s_m with $\texttt{is_required}(s_m) = \text{True}$ are absent in the API parameters. Upon detecting an error, the parser returns fine-grained feedback specifying the issue, allowing the LLM to correct its response.

User Simulator

Evaluating a TOD system ideally requires interactions with real users to assess its effectiveness in goal-oriented scenarios. However, deploying and managing real-user evaluations is expensive and time-consuming. To overcome this challenge, we develop a goal-driven user simulator that can interact with TOD systems in a controlled and scalable manner. An effective user simulator must first accurately convey its needs by specifying the required slot values (e.g., 'departure city") before optionally requesting information (e.g., the flight's arrival time") from the TOD system. To construct such a simulator, we utilize dialog data $\mathcal{T}_{d_x}$ consisting of user goals, expressed through API calls $A = [a_1, a_2, \ldots, a_n]$, and the request slots $R = [s_1, s_2, \ldots, s_m]$ that the user should request. To train the user simulator, we optimize an instruction-finetuned model as:

$$\mathcal{L} = -\sum_{k=1}^{|u_t|} \log p(w_k \mid w_{<k}, H_t, A, R),$$

where w_k denotes the k-th token in the user utterance u_t at turn t, and $w_{<k}$ represents all preceding tokens in the same utterance. The simulator learns to express user goals in natural language by answering requested slots from the TOD system, and requests additional information from the TOD system, conditioned on the set of API calls A, request slots R, and dialog context H_t. To conduct an interactive session between a trained user simulator and the TOD system, the simulator initiates the conversation by retrieving the first user goal a_1 from A and associated request slot s_1 from R. This process continues iteratively until all user goals in A and their associated request slots in R have been processed.

Statistic	SGD	Bitod
Total Dialogs	4,201	352
Total Dialogs (Single-domain)	1,331	111
Total Dialogs (Multi-domain)	2,870	241
Total API Calls	13,239	1,005
Total API Calls (Single-domain)	2,188	127
Total API Calls (Multi-domain)	11,051	878
Total Turns	89,428	6,979
Total User Req. Slots	8,271	500
Avg. API calls per dialog	3.15	2.85
Avg. API calls (Single-domain)	1.64	1.14
Avg. API calls (Multi-domain)	3.85	3.64
Avg. turns per dialog	21.28	19.82
Avg. User Req. Slots	1.96	1.42
Avg. parameters per API call	2.96	3.51
Total Unique API methods	34	7
Total Unique API parameters	88	20

Table 1: Test Dataset Statistics for SGD and BiTOD.

Experiments

Datasets

We conduct our experiments using two datasets: the Schema-Guided dialog (SGD) dataset (Rastogi et al. 2020) and the Bilingual Task-Oriented dialog (BiToD) dataset (Lin et al. 2021b). Since BiToD includes dialogs in both Chinese and English, we retain only the English dialogs for our analysis. Both datasets provide domain-specific schemas along with corresponding dialog conversations, which are essential for baseline models. A comparative summary of key statistics for both datasets is presented in Table 1.

Experimental Setup

We integrated four LLMs in RealTOD: two open-source models, DeepSeek-V3 (Liu et al. 2024) and Llama-3.3-70B-Instruct (Dubey et al. 2024), and two proprietary models, GPT-4o (Achiam et al. 2023) and Claude 3.5 Sonnet (Anthropic 2023). For GPT-4o, we accessed the model via the official OpenAI API, while Claude 3.5 Sonnet was queried using the official Anthropic API.

We fine-tune Flan-T5 model (Chung et al. 2024) to act as a user simulator for each dataset. Specifically, we use the "google/flan-t5-base" model, consisting of 250 million parameters. During fine-tuning, we set the warm-up steps to 100 and applied early stopping with patience of three. The models were trained for 10 epochs.

Evaluation Metrics

To comprehensively evaluate the performance of RealTOD and baseline models, we assess the following: (*i*) Dialog-Level System Response, (*ii*) Dialog-Level Language Diversity, (*iii*) API Call, and (*iv*) Dialog Success Rate.

Dialog-Level System Response. To assess the quality of the responses generated by RealTOD, we removed all user responses produced by our user simulator, retaining only

system responses. We then concatenated all system turns into a single text containing only system-generated outputs. The same process was applied to the ground truth dialog, keeping and concatenating only the system turns. Finally, we evaluated system response quality at the dialog level by comparing the generated responses to the ground truth using BERTScore (Zhang et al. 2019), a metric that measures semantic similarity between texts. Furthermore, we utilize "microsoft/mpnet-base" as the foundational model for computing BERTScore.

Dialog-Level Language Diversity. To assess the lexical diversity of the system responses, we compute Shannon Entropy (SE) and Bigram Conditional Entropy (CE) over the system turns (Terragni et al. 2023; Xu et al. 2024). Following the same preprocessing steps as in the previous section, we remove all user turns and concatenate only the system responses for each dialog. The SE and CE are then computed at the dialog level to quantify the diversity and richness of the language used by the model.

API Calls. To evaluate the quality of API Calls, we first extract the key-value pairs $(name(s_k), v_k)_{k=1}^{n}$, along with method name i from the generated API call using regular expressions. *Method Accuracy* evaluates whether the generated API call uses the correct method name, assessed using exact matching. *Parameter Name Accuracy* determines whether all ground truth key names are included in the generated API call, using fuzzy matching. *Parameter Value Accuracy* verifies if the value associated with a correctly predicted key matches the ground truth, also using fuzzy matching. Notably, this metric is computed only when the corresponding *Parameter Name* is correctly predicted. *Operator Accuracy* applies specifically to the BiToD dataset, as only this dataset includes API calls with operators (e.g., "at_least", "one_of"). We assess this using fuzzy matching. *Full API Accuracy* measures whether the entire API call – including the method, parameter, values, and, for BiToD, the operator – matches the ground truth.

Dialog Success Rate. This metric measures the percentage of dialogs in which all API calls achieve 100% *Full API Accuracy*. In other words, it represents the proportion of dialogs where every generated API call matches the ground truth, ensuring complete correctness throughout the dialog.

Baseline Methods

We compare RealTOD against several strong baselines.

SyncTOD (Saley et al. 2024) improves in-context learning for LLM-powered task-oriented dialog systems by using an auxiliary model to predict hints about the expected response, which guide the selection of proper in-context exemplars.

AutoTOD (Xu et al. 2024) is a zero-shot task-oriented dialog agent that eliminates traditional modules, relying only on instruction-following LLMs like GPT-4. It requires no task-specific training and autonomously decides actions, queries APIs, and generates responses.

ZS-TOD (Mosharrof, Maqbool, and Siddique 2023) is a zero-shot task-oriented dialog system that generalizes to unseen domains using domain schemas instead of memorizing task-specific patterns. It replaces full dialog history with a concise summary, reducing context complexity.

Q-TOD (Tian et al. 2022) is a query-driven task-oriented dialog system that employs a Transformer to generate natural language queries from the dialog context for retrieving relevant knowledge, which is then used to generate system responses.

SOLOIST (Peng et al. 2021) is a Transformer-based task-oriented dialog system that unifies multiple dialog modules into a single pre-trained model. It leverages transfer learning and machine teaching, allowing adaptation to new tasks with minimal labeled data.

SimpleTOD (Hosseini-Asl et al. 2020) treats task-oriented dialog as a single sequence generation problem, using a causal language model to predict dialog state, actions, and responses auto regressively.

Results and Analysis

Here, we focus on the accuracy of API calls generated by the dialog system and the overall quality of system responses at the dialog level. A fine-grained evaluation metric for system responses is provided in Appendix A. For details on user simulator performance, see Appendix C.

Evaluating the Quality of API Calls

Table 2 presents the API call accuracy results on both the SGD and BiToD datasets.

Comparing RealTOD Performance to Baselines. A key observation is that across both datasets, nearly all variants of RealTOD outperform the baseline models across all evaluation metrics, including Method Accuracy, Param Names Accuracy, Param Values Accuracy, Operator Accuracy (for BiToD), and Full API Accuracy. Notably, when focusing on Full API Accuracy, we see substantial gains of RealTOD over baselines. For instance, Claude surpasses AutoTOD, the strongest baseline, by 37.10% in Full API Accuracy on the SGD dataset. Similarly, on BiToD, GPT-4o outperforms SimpleTOD, the best baseline model, by 10.32%, highlighting the robustness of our approach. Moreover, to view the dialogs generated by RealTOD, please refer to Appendix D.

Open-Source vs. Proprietary Models. A notable trend in Table 2 is the consistent superiority of proprietary models (GPT-4o, Claude) over open-source counterparts (DeepSeek, Llama) in terms of Full API Accuracy across both datasets. For instance, on the SGD dataset, Claude outperforms DeepSeek by 20.60%, while on BiTOD, GPT-4o achieves a 31.96% higher Full API Accuracy than Llama. These results underscore the performance gap between proprietary and open-source LLMs.

Model-Specific Observations. Interestingly, when comparing Llama and DeepSeek in the Table 2, their relative performance depends on the dataset. While Llama yields higher accuracies in most metrics on SGD, the trend reverses in BiToD, where DeepSeek significantly outperforms Llama on almost all metrics. We attribute this to DeepSeek's closer alignment with Chinese data, which proves advantageous for BiToD's English subset that still contains Chinese references (e.g., restaurant names). This shows that LLM performance in TOD tasks depends on alignment with the dataset's language and domain.

Dataset	LLM Model	Method Accuracy			Param Names Accuracy			Param Values Accuracy			Operator Accuracy			Full API Accuracy		
		Single	Multi	Both	Single	Multi	Both	Single	Multi	Both	Single	Multi	Both	Single	Multi	Both
SGD	SOLOIST	61.56	64.98	64.41	44.88	47.45	47.02	42.99	45.50	45.08	N/A	N/A	N/A	24.54	26.58	26.24
	Q-TOD	40.08	42.64	42.21	31.91	35.03	34.51	31.35	34.39	33.88				19.24	22.18	21.69
	SimpleTOD	53.51	59.65	58.62	44.32	50.16	49.18	41.92	47.38	46.46				17.55	20.70	21.25
	ZS-TOD	29.11	50.46	46.88	21.52	38.26	35.46	21.52	34.87	32.27				07.67	14.70	13.52
	SyncTOD	30.80	46.93	44.232	27.60	46.26	43.13	26.53	44.57	41.55				16.36	30.92	28.48
	AutoTOD	56.67	62.61	61.61	58.50	65.05	63.95	54.74	61.51	60.37				42.04	48.16	47.14
	RealTOD-GPT-4o	80.60	71.26	72.81	84.72	73.54	75.40	81.44	70.57	72.38	N/A	N/A	N/A	68.71	57.89	59.69
	RealTOD-Claude	**88.29**	**81.74**	**82.83**	**88.91**	**80.59**	**81.97**	**85.66**	**77.08**	**78.51**				**72.58**	**63.04**	**64.63**
	RealTOD-DeepSeek	81.11	69.25	71.22	83.78	70.80	72.96	79.42	66.42	68.58				63.27	51.67	53.59
	RealTOD-Llama	80.32	69.60	71.39	86.65	71.46	73.99	82.36	67.56	70.02				63.23	52.58	54.35
BiTOD	SOLOIST	52.5	69.82	67.97	27.96	61.37	57.81	27.18	59.02	55.63	27.96	61.11	57.58	20.00	50.81	47.53
	Q-TOD	44.09	42.74	42.89	44.56	43.42	43.54	44.54	42.58	42.79	44.56	43.37	43.50	42.23	39.14	39.47
	SimpleTOD	28.57	59.71	56.39	27.61	59.19	55.82	26.83	57.32	54.07	27.61	59.01	55.66	20.49	**50.89**	47.64
	ZS-TOD	26.0	33.53	32.73	23.11	31.55	30.65	23.11	30.23	29.47	22.49	29.86	29.07	16.14	20.32	19.88
	SyncTOD	36.64	31.70	32.23	60.80	41.79	43.84	55.68	38.86	40.67	52.42	37.12	38.77	13.66	17.35	16.95
	AutoTOD	64.29	48.42	49.84	41.73	23.84	25.44	38.45	21.84	23.33	31.48	21.15	22.08	17.86	14.04	14.38
	RealTOD-GPT-4o	82.81	73.82	74.97	82.81	71.20	72.69	82.31	**66.90**	68.88	79.56	70.55	71.71	68.75	50.17	**52.56**
	RealTOD-Claude	**94.49**	66.74	**83.07**	**91.47**	69.06	**79.15**	**90.05**	60.19	**73.34**	**90.81**	**76.38**	**78.22**	**71.65**	47.30	50.40
	RealTOD-DeepSeek	90.55	**75.92**	77.79	86.93	**72.02**	73.92	84.69	65.12	67.62	85.33	71.35	73.13	62.20	44.47	46.73
	RealTOD-Llama	85.16	61.78	64.81	83.46	60.84	63.77	82.43	54.16	57.82	79.27	59.19	61.78	66.41	35.88	39.83

Table 2: API Call Accuracy breakdown across all models on the SGD and BiTOD datasets. Accuracy is reported across multiple metrics, including method, parameter name, parameter value, operator, and overall full API accuracy. Results are shown for single-domain, multi-domain, and both domains.

Comparison Between Metrics. Across both datasets, we observe that all models, including baselines and RealTOD, tend to perform better on Method Accuracy and Parameter Names Accuracy than on Parameter Values Accuracy and Operator Accuracy. This suggests that identifying the correct method or parameter name from the domain schema is generally easier than generating the appropriate value or selecting the correct operator.

Evaluating the Quality of System Responses

So far, we have evaluated the dialog systems primarily based on their ability to generate accurate API calls. However, assessing the overall performance of the dialog systems also requires analyzing the quality of their natural language responses. To this end, we refer to Table 3. This table shows that SyncTOD consistently outperforms other methods in terms of BERTScore (F1) across both the SGD and BiTOD datasets. This is expected, as SyncTOD is designed to produce responses that closely align with the style of the source dataset. Following SyncTOD, the supervised models rank second in BERTScore (F1), which is again unsurprising given that they are trained to mimic the language patterns found in the training data.

However, a high BERTScore does not necessarily indicate that a model generates high-quality responses. It primarily reflects surface-level similarity to reference responses. Therefore, to evaluate the richness of the generated text, we also consider diversity metrics. As shown in Table 3, RealTOD clearly excels in this regard, achieving the highest scores in both Shannon Entropy (SE) and Bigram Conditional Entropy (CE). This suggests that while RealTOD's responses may be slightly less similar to the reference responses (as reflected by its lower BERTScores), they are more varied and diverse. Such diversity is a valuable trait in dialog systems, as it helps prevent repetitive or overly generic replies.

Dataset	LLM Model	BERTScore System (F1)			Language Diversity	
		Single	Multi	Both	SE	CE
SGD	SOLOIST	0.6732	0.6811	0.6732	6.2337	0.8688
	Q-TOD	0.6559	0.7281	0.7049	4.7595	0.5543
	SimpleTOD	0.5963	0.6227	0.6142	5.7914	0.7282
	ZS-TOD	0.5709	0.5748	0.5736	5.5656	0.8516
	SyncTOD	**0.6867**	**0.7566**	**0.7341**	5.5451	0.7403
	AutoTOD	0.6735	0.7080	0.6969	6.4834	1.1753
	RealTOD-GPT-4o	0.6547	0.6544	0.6545	6.3101	1.0610
	RealTOD-Claude	0.6552	0.6694	0.6649	**6.7078**	1.3033
	RealTOD-DeepSeek	0.6345	0.6384	0.6372	6.5418	1.0674
	RealTOD-Llama	0.6019	0.5979	0.5992	6.4796	**1.4515**
BiTOD	SOLOIST	0.5692	0.7876	0.7186	5.7138	0.8909
	Q-TOD	0.6481	0.7356	0.7077	4.8532	0.6192
	SimpleTOD	0.5781	**0.7964**	0.7271	5.4943	0.8033
	ZS-TOD	0.6563	0.7467	0.7180	5.7151	0.7669
	SyncTOD	**0.6936**	0.7552	**0.7355**	6.4828	1.0154
	AutoTOD	0.6526	0.6745	0.6675	6.5369	1.1007
	RealTOD-GPT-4o	0.6543	0.6447	0.6477	6.4121	1.0897
	RealTOD-Claude	0.6523	0.6392	0.6434	**6.8699**	1.3602
	RealTOD-DeepSeek	0.6454	0.6278	0.6334	6.5743	1.0689
	RealTOD-Llama	0.5927	0.5694	0.5769	6.5957	**1.4415**

Table 3: Comparison of RealTOD with baseline models for Language Diversity and BERTScore for system response on SGD and BiTOD datasets.

Ablation Study

To assess the effectiveness of our proposed components, Fine-Grained Feedback and Prompt Chaining, we conducted an ablation study using 100 dialog conversations sampled from the SGD dataset (50 from multi-domain and 50 from single-domain). We evaluated all four variants of RealTOD (GPT-4o, Claude, Llama, and DeepSeek) under four different settings: one without either of the components, one with Fine-Grained Feedback only, one with Prompt Chaining only, and one with both components. Moreover, we used the Full API Accuracy as our comparison metric. This experimental design allowed us to isolate the impact of each component and determine

Fine-Grained Feedback	Prompt Chaining	GPT-4o	Claude	DeepSeek	Llama
✗	✗	51.89	29.92	45.83	49.62
✓	✗	56.06	36.74	48.10	57.19
✗	✓	64.01	70.45	54.92	59.84
✓	✓	**66.66**	**72.34**	**59.46**	**63.63**

Table 4: Ablation study of full API accuracy on the SGD dataset to evaluate the impact of fine-grained feedback and prompt chaining across different LLMs.

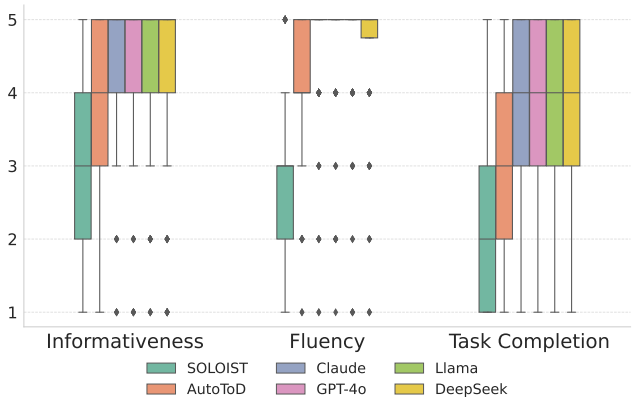

Figure 2: Human Evaluation Results on SGD and BiTOD. Evaluators rated generated conversations from 1 to 5 based on three aspects: informativeness, fluency, and task completion.

their individual and combined contributions to performance. The results are provided in the Table 4. As it can be seen in Table 4, adding Fine-Grained Feedback alone leads to moderate improvements, indicating its role in refining APIs. Prompt Chaining, on the other hand, provides a more substantial boost. The combination of both components yields the highest accuracy, demonstrating their complementary nature.

Human Evaluation

We conducted a human evaluation using Amazon Mechanical Turk to assess the performance of our models. As baselines, we selected SOLOIST and AutoTOD, based on their good performance in Table 2, and compared them against all four variants of RealTOD. For our evaluation, we sampled 100 dialogs from the test sets of our chosen datasets (SGD and BiToD), with 50 each from single and multi-domain tasks. We asked the human evaluators to rate the generated conversations on a scale of 1 to 5 across three key aspects: *Informativeness*, *Fluency*, and *Task Completion Rate*. Figure 2 shows the human evaluation results, where all four variants of RealTOD outperformed the baseline models (SOLOIST and AutoToD), supporting the reliability of our evaluation metrics.

Dialog Success Rate

To rigorously assess the quality of the generated dialogs, we conducted an experiment to measure the dialog success rate as the number of API calls within a dialog increases. As shown in Figure 3, all variants of RealTOD exhibit a declining trend in dialog success rate as the number of API calls increases. This trend is consistent across both the SGD and BiTOD datasets. The primary reason for this decline is the interdependence of API calls. For example, when a user books a restaurant at a particular destination, the same location is often referenced for booking a taxi or later searching for nearby hotels. Any errors in earlier API calls can propagate, making subsequent calls more prone to failure. The Figure 3 highlights a limitation of LLM-powered TOD systems, suggesting that they are still far from achieving perfect performance. Further research is needed to enhance their ability to handle these scenarios more effectively.

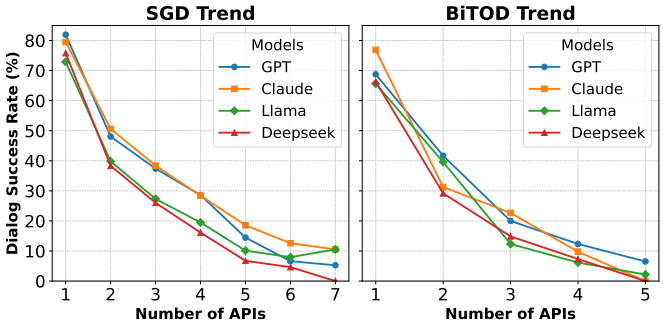

Figure 3: Dialog success rate declines as the number of API calls increases across models on the SGD and BiTOD datasets. This trend underscores the challenge of error propagation in LLM-powered TOD systems, where early mistakes adversely affect later interactions.

Conclusion

We presented RealTOD, a task-completion-centric framework for TOD systems that eliminates the need for domain-specific fine-tuning and enables reliable performance across diverse domains. Our work tackles a fundamental limitation of current LLM-based TOD systems: their inability to consistently complete tasks due to unreliable API call generation, despite their fluency in natural language responses. RealTOD introduces three key innovations: (*1*) a prompt chaining mechanism that enables zero-shot generalization to new tasks and domains by automatically generating schema-aligned in-context demonstrations; (*2*) a fine-grained feedback loop that verifies each API call against the domain schema and provides targeted prompts to guide correction, significantly improving task execution reliability; and (*3*) a task-centric evaluation metric that goes beyond traditional n-gram-based metrics by precisely assessing task completion through full API call accuracy and detailed error breakdowns. Through comprehensive experiments across two benchmark datasets and four LLMs, we demonstrate that RealTOD achieves substantial gains in task success rates, setting a new standard for LLM-based TOD systems. Beyond these performance improvements, our analysis reveals persistent challenges in dialog-based task completion, including domain coverage errors in multi-domain settings, failures in long-horizon planning across turns, and misalignment when integrating external search results, highlighting directions for future research.

References

Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Agarwal, R.; Singh, A.; Zhang, L. M.; Bohnet, B.; Rosias, L.; Chan, S. C.; Zhang, B.; Faust, A.; and Larochelle, H. 2024. Many-shot In-Context Learning. In *ICML 2024 Workshop on In-Context Learning*.

Anthropic. 2023. Claude. https://www.anthropic.com/claude. Accessed: 2024-03-01.

Bai, H.; Zhou, Y.; Cemri, M.; Pan, J.; Suhr, A.; Levine, S.; and Kumar, A. 2024. DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning. *arXiv preprint arXiv:2406.11896*.

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901.

Chen, W.; Chen, J.; Qin, P.; Yan, X.; and Wang, W. Y. 2019. Semantically Conditioned Dialog Response Generation via Hierarchical Disentangled Self-Attention. In Korhonen, A.; Traum, D.; and Màrquez, L., eds., *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3696–3709. Florence, Italy: Association for Computational Linguistics.

Cheng, Q.; Li, L.; Quan, G.; Gao, F.; Mou, X.; and Qiu, X. 2022. Is MultiWOZ a Solved Task? An Interactive TOD Evaluation Framework with User Simulator. In Goldberg, Y.; Kozareva, Z.; and Zhang, Y., eds., *Findings of the Association for Computational Linguistics: EMNLP 2022*, 1248–1259. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics.

Chung, H. W.; Hou, L.; Longpre, S.; Zoph, B.; Tay, Y.; Fedus, W.; Li, Y.; Wang, X.; Dehghani, M.; Brahma, S.; et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70): 1–53.

Chung, W.; Cahyawijaya, S.; Wilie, B.; Lovenia, H.; and Fung, P. 2023. InstructTODS: Large Language Models for End-to-End Task-Oriented Dialogue Systems. In Chen, K.; and Ku, L.-W., eds., *Proceedings of the Second Workshop on Natural Language Interfaces*, 1–21. Bali, Indonesia: Association for Computational Linguistics.

Cui, M.; Gao, P.; Liu, W.; Luan, J.; and Wang, B. 2025. Multilingual Machine Translation with Open Large Language Models at Practical Scale: An Empirical Study. In Chiruzzo, L.; Ritter, A.; and Wang, L., eds., *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 5420–5443. Albuquerque, New Mexico: Association for Computational Linguistics. ISBN 979-8-89176-189-6.

Davidson, S.; Romeo, S.; Shu, R.; Gung, J.; Gupta, A.; Mansour, S.; and Zhang, Y. 2023. User simulation with large language models for evaluating task-oriented dialogue. *arXiv preprint arXiv:2309.13233*.

Dong, Q.; Li, L.; Dai, D.; Zheng, C.; Ma, J.; Li, R.; Xia, H.; Xu, J.; Wu, Z.; Chang, B.; Sun, X.; Li, L.; and Sui, Z. 2024. A Survey on In-context Learning. In Al-Onaizan, Y.; Bansal, M.; and Chen, Y.-N., eds., *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 1107–1128. Miami, Florida, USA: Association for Computational Linguistics.

Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Eckert, W.; Levin, E.; and Pieraccini, R. 1997. User modeling for spoken dialogue system evaluation. In *1997 IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*, 80–87.

Fereidouni, M.; Mosharrof, A.; and Siddique, A. 2024. Grounded Language Agent for Product Search via Intelligent Web Interactions. In Kumar, S.; Balachandran, V.; Park, C. Y.; Shi, W.; Hayati, S. A.; Tsvetkov, Y.; Smith, N.; Hajishirzi, H.; Kang, D.; and Jurgens, D., eds., *Proceedings of the 1st Workshop on Customizable NLP: Progress and Challenges in Customizing NLP for a Domain, Application, Group, or Individual (CustomNLP4U)*, 63–75. Miami, Florida, USA: Association for Computational Linguistics.

Furuta, H.; Lee, K.-H.; Nachum, O.; Matsuo, Y.; Faust, A.; Gu, S. S.; and Gur, I. 2024. Multimodal Web Navigation with Instruction-Finetuned Foundation Models. In *The Twelfth International Conference on Learning Representations*.

Gain, B.; Bandyopadhyay, D.; and Ekbal, A. 2025. Bridging the linguistic divide: A survey on leveraging large language models for machine translation. *arXiv preprint arXiv:2504.01919*.

Gao, J.; Xiang, L.; Wu, H.; Zhao, H.; Tong, Y.; and He, Z. 2023. An Adaptive Prompt Generation Framework for Task-oriented Dialogue System. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Findings of the Association for Computational Linguistics: EMNLP 2023*, 1078–1089. Singapore: Association for Computational Linguistics.

Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

He, W.; Dai, Y.; Hui, B.; Yang, M.; Cao, Z.; Dong, J.; Huang, F.; Si, L.; and Li, Y. 2022. SPACE-2: Tree-Structured Semi-Supervised Contrastive Pre-training for Task-Oriented Dialog Understanding. In Calzolari, N.; Huang, C.-R.; Kim, H.; Pustejovsky, J.; Wanner, L.; Choi, K.-S.; Ryu, P.-M.; Chen, H.-H.; Donatelli, L.; Ji, H.; Kurohashi, S.; Paggio, P.; Xue, N.; Kim, S.; Hahm, Y.; He, Z.; Lee, T. K.; Santus, E.; Bond, F.; and Na, S.-H., eds., *Proceedings of the 29th International Conference on Computational Linguistics*, 553–569. Gyeongju, Republic of Korea: International Committee on Computational Linguistics.

Hosseini-Asl, E.; McCann, B.; Wu, C.-S.; Yavuz, S.; and Socher, R. 2020. A simple language model for task-oriented dialogue. *Advances in Neural Information Processing Systems*, 33: 20179–20191.

Hudeček, V.; and Dusek, O. 2023. Are Large Language Models All You Need for Task-Oriented Dialogue? In Stoyanchev, S.; Joty, S.; Schlangen, D.; Dusek, O.; Kennington, C.; and Alikhani, M., eds., *Proceedings of the 24th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 216–228. Prague, Czechia: Association for Computational Linguistics.

Imrattanatrai, W.; and Fukuda, K. 2023. End-to-End Task-Oriented Dialogue Systems Based on Schema. In Rogers, A.; Boyd-Graber, J.; and Okazaki, N., eds., *Findings of the Association for Computational Linguistics: ACL 2023*, 10148–10161. Toronto, Canada: Association for Computational Linguistics.

Jain, N.; Kwiatkowski, R.; Ray, B.; Ramanathan, M. K.; and Kumar, V. 2024. On Mitigating Code LLM Hallucinations with API Documentation. *arXiv preprint arXiv:2407.09726*.

Kim, B.; Jang, Y.; Logeswaran, L.; Kim, G.-H.; Kim, Y. J.; Lee, H.; and Lee, M. 2024. Prospector: Improving LLM Agents with Self-Asking and Trajectory Ranking. In Al-Onaizan, Y.; Bansal, M.; and Chen, Y.-N., eds., *Findings of the Association for Computational Linguistics: EMNLP 2024*, 14958–14976. Miami, Florida, USA: Association for Computational Linguistics.

Labruna, T.; Brenna, S.; Zaninello, A.; and Magnini, B. 2023. Unraveling ChatGPT: A Critical Analysis of AI-Generated Goal-Oriented Dialogues and Annotations. In *AI*IA*, 151–171.

Lee, C.; Jung, S.; Kim, S.; and Lee, G. G. 2009. Example-based dialog modeling for practical multi-domain dialog system. *Speech Communication*, 51(5): 466–484.

Lee, S. 2013. Structured Discriminative Model For Dialog State Tracking. In Eskenazi, M.; Strube, M.; Di Eugenio, B.; and Williams, J. D., eds., *Proceedings of the SIGDIAL 2013 Conference*, 442–451. Metz, France: Association for Computational Linguistics.

Lee, S.; Choi, J.; Lee, J.; Wasi, M. H.; Choi, H.; Ko, S. Y.; Oh, S.; and Shin, I. 2023. Explore, select, derive, and recall: Augmenting llm with human-like memory for mobile task automation. *arXiv preprint arXiv:2312.03003*.

Lei, W.; Jin, X.; Kan, M.-Y.; Ren, Z.; He, X.; and Yin, D. 2018. Sequicity: Simplifying Task-oriented Dialogue Systems with Single Sequence-to-Sequence Architectures. In Gurevych, I.; and Miyao, Y., eds., *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1437–1447. Melbourne, Australia: Association for Computational Linguistics.

Lin, C.-Y. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, 74–81. Barcelona, Spain: Association for Computational Linguistics.

Lin, H.-c.; Geishauser, C.; Feng, S.; Lubis, N.; van Niekerk, C.; Heck, M.; and Gasic, M. 2022. GenTUS: Simulating User Behaviour and Language in Task-oriented Dialogues with Generative Transformers. In Lemon, O.; Hakkani-Tur, D.; Li, J. J.; Ashrafzadeh, A.; Garcia, D. H.; Alikhani, M.; Vandyke, D.; and Dušek, O., eds., *Proceedings of the 23rd Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 270–282. Edinburgh, UK: Association for Computational Linguistics.

Lin, H.-c.; Lubis, N.; Hu, S.; van Niekerk, C.; Geishauser, C.; Heck, M.; Feng, S.; and Gasic, M. 2021a. Domain-independent User Simulation with Transformers for Task-oriented Dialogue Systems. In Li, H.; Levow, G.-A.; Yu, Z.; Gupta, C.; Sisman, B.; Cai, S.; Vandyke, D.; Dethlefs, N.; Wu, Y.; and Li, J. J., eds., *Proceedings of the 22nd Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 445–456. Singapore and Online: Association for Computational Linguistics.

Lin, Z.; Madotto, A.; Winata, G. I.; and Fung, P. 2020. MinTL: Minimalist Transfer Learning for Task-Oriented Dialogue Systems. In Webber, B.; Cohn, T.; He, Y.; and Liu, Y., eds., *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3391–3405. Online: Association for Computational Linguistics.

Lin, Z.; Madotto, A.; Winata, G. I.; Xu, P.; Jiang, F.; Hu, Y.; Shi, C.; and Fung, P. 2021b. BiToD: A Bilingual Multi-Domain Dataset For Task-Oriented Dialogue Modeling. *arXiv preprint arXiv:2106.02787*.

Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; Ruan, C.; et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.

Liu, H.; Cai, Y.; Ou, Z.; Huang, Y.; and Feng, J. 2022a. A Generative User Simulator with GPT-based Architecture and Goal State Tracking for Reinforced Multi-Domain Dialog Systems. In Ou, Z.; Feng, J.; and Li, J., eds., *Proceedings of the Towards Semi-Supervised and Reinforced Task-Oriented Dialog Systems (SereTOD)*, 85–97. Abu Dhabi, Beijing (Hybrid): Association for Computational Linguistics.

Liu, J.; Shen, D.; Zhang, Y.; Dolan, B.; Carin, L.; and Chen, W. 2022b. What Makes Good In-Context Examples for GPT-3? In Agirre, E.; Apidianaki, M.; and Vulić, I., eds., *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, 100–114. Dublin, Ireland and Online: Association for Computational Linguistics.

Ma, K.; Zhang, H.; Wang, H.; Pan, X.; and Yu, D. 2023. LASER: LLM Agent with State-Space Exploration for Web Navigation. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*.

Madotto, A.; Wu, C.-S.; and Fung, P. 2018. Mem2Seq: Effectively Incorporating Knowledge Bases into End-to-End Task-Oriented Dialog Systems. In Gurevych, I.; and Miyao, Y., eds., *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1468–1478. Melbourne, Australia: Association for Computational Linguistics.

Mi, F.; Wang, Y.; and Li, Y. 2022. CINS: Comprehensive Instruction for Few-Shot Learning in Task-Oriented Dialog Systems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(10): 11076–11084.

Mosharrof, A.; Maqbool, M. H.; and Siddique, A. 2023. Zero-Shot Generalizable End-to-End Task-Oriented Dialog System using Context Summarization and Domain Schema. *arXiv preprint arXiv:2303.16252*.

Naveed, H.; Khan, A. U.; Qiu, S.; Saqib, M.; Anwar, S.; Usman, M.; Akhtar, N.; Barnes, N.; and Mian, A. 2023. A comprehensive overview of large language models. *arXiv preprint arXiv:2307.06435*.

Nekvinda, T.; and Dušek, O. 2021. Shades of BLEU, Flavours of Success: The Case of MultiWOZ. In Bosselut, A.; Durmus, E.; Gangal, V. P.; Gehrmann, S.; Jernite, Y.; Perez-Beltrachini, L.; Shaikh, S.; and Xu, W., eds., *Proceedings of the 1st Workshop on Natural Language Generation, Evaluation, and Metrics (GEM 2021)*, 34–46. Online: Association for Computational Linguistics.

OpenAI. ???? o3. [Large Language Model].

Papineni, K.; Roukos, S.; Ward, T.; and Zhu, W.-J. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In Isabelle, P.; Charniak, E.; and Lin, D., eds., *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311–318. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics.

Peng, B.; Li, C.; Li, J.; Shayandeh, S.; Liden, L.; and Gao, J. 2021. Soloist: Building task bots at scale with transfer learning and machine teaching. *Transactions of the Association for Computational Linguistics*, 9: 807–824.

Peng, B.; Zhu, C.; Li, C.; Li, X.; Li, J.; Zeng, M.; and Gao, J. 2020. Few-shot Natural Language Generation for Task-Oriented Dialog. In Cohn, T.; He, Y.; and Liu, Y., eds., *Findings of the Association for Computational Linguistics: EMNLP 2020*, 172–182. Online: Association for Computational Linguistics.

Pu, X.; Gao, M.; and Wan, X. 2023. Summarization is (Almost) Dead. *ArXiv*, abs/2309.09558.

Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language Models are Unsupervised Multitask Learners.

Rastogi, A.; Zang, X.; Sunkara, S.; Gupta, R.; and Khaitan, P. 2020. Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, 8689–8696.

Saley, V. V.; Das, R. J.; Raghu, D.; and ., M. 2024. Synergizing In-context Learning with Hints for End-to-end Task-oriented Dialog Systems. In Al-Onaizan, Y.; Bansal, M.; and Chen, Y.-N., eds., *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 5596–5612. Miami, Florida, USA: Association for Computational Linguistics.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23. Red Hook, NY, USA: Curran Associates Inc.

Shu, L.; Luo, L.; Hoskere, J.; Zhu, Y.; Liu, Y.; Tong, S.; Chen, J.; and Meng, L. 2024. Rewritelm: An instruction-tuned large language model for text rewriting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 18970–18980.

Siddique, A.; Maqbool, M.; Taywade, K.; and Foroosh, H. 2022. Personalizing Task-oriented Dialog Systems via Zero-shot Generalizable Reward Function. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, CIKM '22, 1787–1797. New York, NY, USA: Association for Computing Machinery. ISBN 9781450392365.

Song, Y.; Lothritz, C.; Tang, X.; Ezzini, S.; Klein, J.; Bissyandé, T. F.; Boytsov, A.; Ble, U.; and Goujon, A. 2025. CallNavi: A Study and Challenge on Function Calling Routing and Invocation in Large Language Models. *arXiv preprint arXiv:2501.05255*.

Sridhar, A.; Lo, R.; Xu, F. F.; Zhu, H.; and Zhou, S. 2023. Hierarchical Prompting Assists Large Language Model on Web Navigation. In *Conference on Empirical Methods in Natural Language Processing*.

Su, Y.; Shu, L.; Mansimov, E.; Gupta, A.; Cai, D.; Lai, Y.-A.; and Zhang, Y. 2022. Multi-Task Pre-Training for Plug-and-Play Task-Oriented Dialogue System. In Muresan, S.; Nakov, P.; and Villavicencio, A., eds., *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4661–4676. Dublin, Ireland: Association for Computational Linguistics.

Sun, X.; Li, X.; Li, J.; Wu, F.; Guo, S.; Zhang, T.; and Wang, G. 2023. Text Classification via Large Language Models. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8990–9005. Singapore: Association for Computational Linguistics.

Team, Q. 2024. Qwen 2.5 MAX: A Large Language Model for Advanced Natural Language Processing. https://www.example.com/qwen-max. Accessed: 2024-03-01.

Terragni, S.; Filipavicius, M.; Khau, N.; Guedes, B.; Manso, A.; and Mathis, R. 2023. In-context learning user simulators for task-oriented dialog systems. *arXiv preprint arXiv:2306.00774*.

Thoppilan, R.; Freitas, D. D.; Hall, J.; Shazeer, N. M.; Kulshreshtha, A.; Cheng, H.-T.; Jin, A.; Bos, T.; Baker, L.; Du, Y.; Li, Y.; Lee, H.; Zheng, H. S.; Ghafouri, A.; Menegali, M.; Huang, Y.; Krikun, M.; Lepikhin, D.; Qin, J.; Chen, D.; Xu, Y.; Chen, Z.; Roberts, A.; Bosma, M.; Zhou, Y.; Chang, C.-C.; Krivokon, I. A.; Rusch, W. J.; Pickett, M.; Meier-Hellstern, K. S.; Morris, M. R.; Doshi, T.; Santos, R. D.; Duke, T.; Søraker, J. H.; Zevenbergen, B.; Prabhakaran, V.; Díaz, M.; Hutchinson, B.; Olson, K.; Molina, A.; Hoffman-John, E.; Lee, J.; Aroyo, L.; Rajakumar, R.; Butryna, A.; Lamm, M.; Kuzmina, V. O.; Fenton, J.; Cohen, A.; Bernstein, R.; Kurzweil, R.; Aguera-Arcas, B.; Cui, C.; Croak, M. R.; Chi, E. H.; and Le, Q. 2022. LaMDA: Language Models for Dialog Applications. *ArXiv*, abs/2201.08239.

Tian, X.; Lin, Y.; Song, M.; Bao, S.; Wang, F.; He, H.; Sun, S.; and Wu, H. 2022. Q-TOD: A Query-driven Task-oriented Dialogue System. In Goldberg, Y.; Kozareva, Z.; and Zhang, Y., eds., *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 7260–7271. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics.

Tseng, B.-H.; Dai, Y.; Kreyssig, F.; and Byrne, B. 2021. Transferable Dialogue Systems and User Simulators. In Zong, C.; Xia, F.; Li, W.; and Navigli, R., eds., *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 152–166. Online: Association for Computational Linguistics.

Van Veen, D.; Van Uden, C.; Blankemeier, L.; Delbrouck, J.-B.; Aali, A.; Bluethgen, C.; Pareek, A.; Polacin, M.; Reis, E. P.; Seehofnerova, A.; et al. 2023. Clinical text summarization: adapting large language models can outperform human experts. *Research Square*.

Wang, J.; Xu, H.; Jia, H.; Zhang, X.; Yan, M.; Shen, W.; Zhang, J.; Huang, F.; and Sang, J. 2024a. Mobile-Agent-v2: Mobile Device Operation Assistant with Effective Navigation via Multi-Agent Collaboration. *arXiv preprint arXiv:2406.01014*.

Wang, J.; Xu, H.; Ye, J.; Yan, M.; Shen, W.; Zhang, J.; Huang, F.; and Sang, J. 2024b. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*.

Wang, L.; Lyu, C.; Ji, T.; Zhang, Z.; Yu, D.; Shi, S.; and Tu, Z. 2023. Document-Level Machine Translation with Large Language Models. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 16646–16661. Singapore: Association for Computational Linguistics.

Wang, Z.; Pang, Y.; and Lin, Y. 2023. Large language models are zero-shot text classifiers. *arXiv preprint arXiv:2312.01044*.

Wen, H.; Li, Y.; Liu, G.; Zhao, S.; Yu, T.; Li, T. J.-J.; Jiang, S.; Liu, Y.; Zhang, Y.; and Liu, Y. 2024. AutoDroid: LLM-powered Task Automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, ACM MobiCom '24, 543–557. New York, NY, USA: Association for Computing Machinery. ISBN 9798400704895.

Wen, H.; Wang, H.; Liu, J.; and Li, Y. 2023. Droidbot-gpt: Gpt-powered ui automation for android. *arXiv preprint arXiv:2304.07061*.

Williams, J. D.; and Young, S. 2007. Partially observable Markov decision processes for spoken dialog systems. *Computer Speech & Language*, 21(2): 393–422.

Xu, H.-D.; Mao, X.-L.; Yang, P.; Sun, F.; and Huang, H.-Y. 2024. Rethinking Task-Oriented Dialogue Systems: From Complex Modularity to Zero-Shot Autonomous Agent. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2748–2763.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems*, volume 35, 20744–20757. Curran Associates, Inc.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K. Q.; and Artzi, Y. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.

Zhang, T.; Ladhak, F.; Durmus, E.; Liang, P.; McKeown, K.; and Hashimoto, T. B. 2024a. Benchmarking Large Language Models for News Summarization. *Transactions of the Association for Computational Linguistics*, 12: 39–57.

Zhang, Y.; Wang, M.; Ren, C.; Li, Q.; Tiwari, P.; Wang, B.; and Qin, J. 2024b. Pushing The Limit of LLM Capacity for Text Classification. *arXiv preprint arXiv:2402.07470*.

Dataset	LLM Model	Inform Accuracy (%)		
		Single	Multi	Both
SGD	SOLOIST	44.54	54.42	52.66
	Q-TOD	25.28	26.75	26.49
	SimpleTOD	32.15	51.11	47.74
	ZS-TOD	12.32	11.20	11.40
	SyncTOD	75.00	73.12	73.45
	AutoTOD	76.34	75.84	75.93
	RealTOD-GPT-4o	**82.93**	**76.89**	**78.03**
	RealTOD-Claude	79.44	71.37	72.85
	RealTOD-DeepSeek	82.81	75.45	76.88
	RealTOD-Llama	73.49	70.26	70.88
BiTOD	SOLOIST	80.00	69.42	70.62
	Q-TOD	35.00	26.11	27.11
	SimpleTOD	85.00	71.33	72.88
	ZS-TOD	60.00	66.24	65.53
	SyncTOD	87.50	80.25	81.07
	AutoTOD	**97.50**	**84.39**	**85.87**
	RealTOD-GPT-4o	85.18	79.78	80.26
	RealTOD-Claude	64.70	63.51	63.61
	RealTOD-DeepSeek	86.66	79.25	79.93
	RealTOD-Llama	64.28	65.06	65.00

Table 5: Comparison of Inform Accuracy (%) on SGD and BiTOD datasets across all baseline models and RealTOD.

A Inform Accuracy

Inform Accuracy Metric. To evaluate how effectively RealTOD informs the user about the requested slots, we implemented a regex-based system. First, we identify the slots requested by the user and extract their corresponding values from the search results. Then, we use regex matching to determine whether the system's subsequent responses include those extracted values. If a system turn contains the requested slot values, we consider the system to have successfully provided the required information.

Analysis on Inform Accuracy Results. As expected, Table 5 shows that LLM-powered models perform strongly on the Inform Accuracy metric. On the SGD dataset, our method RealTOD-GPT-4o achieves the highest Inform Accuracy across the Single, Multi, and Both domain settings, with RealTOD-DeepSeek ranking second. On the BiToD dataset, however, AutoTOD outperforms all other models. These results reinforce that LLMs, especially when supported by a dedicated system architecture such as RealTOD, are highly effective at accurately providing requested slot values.

B Limitations

We restricted our experiments to four popular LLMs (GPT-4o, Claude 3.5 Sonnet, DeepSeek-V3, and Llama-3.3-70B-Instruct) due to time and computational constraints. Given the rapid pace of model development (systems such as DeepSeek R1 (Guo et al. 2025), OpenAI o3-mini (OpenAI), and Qwen-2.5 Max (Team 2024), it remains an open question how RealTOD would perform with these newer LLMs.

Dataset	LLM Model	BERTScore User (F1)			BERTScore Overall (F1)		
		Single	Multi	Both	Single	Multi	Both
SGD	RealTOD-GPT-4o	0.6879	0.6712	0.6765	0.6821	0.6761	0.6780
	RealTOD-Claude	0.7053	0.6972	0.6997	0.6822	0.6852	0.6843
	RealTOD-DeepSeek	0.6844	0.6700	0.6746	0.6600	0.6554	0.6569
	RealTOD-Llama	0.6853	0.6766	0.6794	0.6365	0.6256	0.6291
BiTOD	RealTOD-GPT-4o	0.6794	0.6549	0.6627	0.6889	0.6671	0.6741
	RealTOD-Claude	0.6963	0.6737	0.6809	0.6841	0.6625	0.6693
	RealTOD-DeepSeek	0.6979	0.6502	0.6653	0.6789	0.6530	0.6612
	RealTOD-Llama	0.6534	0.6370	0.6423	0.6345	0.5990	0.6104

Table 6: Performance of the Dialog-Level User Simulator and Entire Conversation.

Moreover, while the user simulators fine-tuned in this study perform reasonably well, they are not perfect and may occasionally struggle to respond accurately to RealTOD. Further details on their performance can be found in the next Appendix C.

C User Simulator Performance

To evaluate the performance of our user simulator, we computed BERTScore on user turns (BERTScore User) and on the concatenation of both user and system turns (BERTScore Overall). As shown in Table 6, most BERTScores fall within the range of approximately 0.6 to 0.7, showing that the simulated user responses maintain a reasonable level of similarity to the reference.

D Example Dialog Responses

Tables 7, 8, 9, and 10 present an example of a multi-domain conversation where our User Simulator interacts with different variants of RealTOD (Claude, GPT-4o, DeepSeek, and Llama) to check the weather, schedule a property visit, and reserve a car. Additionally, Tables 11 and 12 showcase examples generated by the two baseline models SOLOIST and ZS-TOD. Both of these models struggle with task completion; SOLOIST, on the second turn, missed the API call. The same thing happened when ZS-TOD completely forgot to make the API call when handling a car reservation, ultimately failing to provide the requested information. For model errors, we use ✗, and for correct API calls, we use ✓.

Instruction Prompt Template P_1

Task Description:
Your task is to generate a dialog conversation between a User and a System based on a given domain schema. I will provide a **Schema** for **{domain_X}**, which defines the structure and relevant entities, along with a corresponding dialog conversation for reference. Your goal is to analyze the relationship between the dialog and the schema and then generate a coherent and contextually appropriate dialog conversation for **{domain_Y}** while maintaining consistency with its schema.

Here is a **Schema** for the **{domain_X}**
service_name: {domain_X}
Intents
  intent_no.
  name: IntentName
  is_transactional: True/False
  required_slots: required slot 1, required slot 2, required slot 3, ...
  optional_slots: optional slot 2, optional slot 2, optional slot 3, ...
Slots
    slot_name: slot name 1, slot name 2, slot name 3, ...
    possible_values: value 1, value 2, value 3, ...
end of schema for {domain_X}
A sample **« Dialog Conversation »** between a System and a User will be fetched.

Now, understand the above conversation structure between a User and a System. You will be given a new **Schema** for **{domain_Y}**. You have to generate a full-fledged conversation for the new domain that will be structured like the example above.

Here is a **Schema** for the **{domain_Y}**
service_name: {domain_Y}
Intents
  intent_no.
  name: IntentName
  is_transactional: True/False
  required_slots: required slot 1, required slot 2, required slot 3, ...
  optional_slots: optional slot 2, optional slot 2, optional slot 3, ...
Slots
    slot_name: slot name 1, slot name 2, slot name 3, ...
    possible_values: value 1, value 2, value 3, ...
end of schema for {domain_Y}

Based on the above instructions and example conversation from the domain_X, learn how to generate the full conversation for the new domain_Y domain.
End of Instructions.

Instruction Prompt Template P_2

Task Description:
Think of yourself as an expert chat assistant specialized in the **{domain_name}** domain. Your task is to generate the most natural and helpful responses for a given task-oriented dialog context. I will provide **Schema** for **{domain_name}**, one sample conversation between a System and a User, optionally, search results from the database. Understand the dialog relation to **Schema**. You can request slot values from the User to fulfill the User's current **intent**. Remember that required **slots** are more important than optional **slots**. When making **API** calls, use column names from the **Schema** as parameters. Match the required and optional **slots** with the column names and use them in **API** calls. Before making the call, ensure you've gathered all required **slots** from the User. You can skip unnecessary parameters.

Here is a **Schema** for the **domain_name**
service_name: {domain_name}
Intents
  intent_no.
  name: IntentName
  is_transactional: True/False
  required_slots: required slot 1, required slot 2, required slot 3, ...
  optional_slots: optional slot 2, optional slot 2, optional slot 3, ...
Slots
    slot_name: slot name 1, slot name 2, slot name 3, ...
    possible_values: value 1, value 2, value 3, ...
end of schema
A sample **« `Dialog Conversation` »** between a System and a User will be fetched.

Understand the above structure of conversation between a User and a System. Learn how to interact with the User and generate the most human-like conversational response to the User's **intent**. You may need to make **API** Calls and use the **API** Call results. Based on the above instructions and examples from the **{domain_name}** domain, learn how to interact with a User to generate the most human-like conversational response to the User's current **intent**.
End of Instructions.

Here are a few general **Guidelines** to follow:
- Please avoid asking for too many **slots** in one turn; ideally, ask one slot at a time.
- Don't overwhelm the User with too many questions or choices in one turn.
- Confirm the slot values with the User before finalizing the **API** Call.
- Follow the structure of **API** Call from the above example whenever you are making an **API** Call.
- If you're unsure about something, it's always better to ask or confirm with the User.
- Do not provide all the information in the search results to the User. Provide details only if the User requests them.
- If you feel the User is confused, guide the User with relevant suggestions and ensure it is relevant to their current **intent**.
- You generate only one system response at a time and do not produce search results yourself; search results will be provided to you.

Conversation history: « conversation history » up to turn t will be fetched

Table 7: Multi-Domain SGD Conversation - RealTOD (Claude 3.5 Sonnet)

SGD Multi-Domain Conversation (Claude 3.5 Sonnet)
Goal: User is trying to check the weather in Vancouver, then schedule a property visit at the Golf Club Manor Apartments, and reserve a Hatchback car
User: What's the weather going to be like in Vancouver, BC on March 2nd?
RealTOD: I'll help you check the weather in Vancouver, BC for March 2nd. APICall(method='GetWeather', parameters={ city: Vancouver, date: 2024-03-02 }) ✓
Search Results: [{'city': 'Vancouver', 'date': '2024-03-02', 'temperature': 68, 'precipitation': 25, 'humidity': 26, 'wind': 6}]
RealTOD: On March 2nd in Vancouver, the temperature will be 68 degrees Fahrenheit with a 25 percent chance of precipitation. The humidity will be at 26% with wind speeds of 6 mph. **User:** Okay, sounds good. I'd like to schedule a visit to the Golf Club Manor Apartments. **RealTOD:** I can help you schedule a property visit to Golf Club Manor Apartments. I notice you were checking the weather for March 2nd. Would you like to schedule the visit for that same date? **User:** Yes, I'd like to schedule the visit.
RealTOD: APICall(method='ScheduleVisit', parameters={ property_name: Golf Club Manor Apartments, visit_date: 2024-03-02 }) ✓
Search Results: [{'property_name': 'Golf Club Manor Apartments', 'address': '375 Industrial Parkway # 314', 'area': 'Hayward', 'price': 2500000, 'phone_number': '5105810911'}]
RealTOD: I've confirmed your visit to Golf Club Manor Apartments for March 2nd. The property is located at 375 Industrial Parkway # 314 in Hayward. Would you like to know any specific details about the property? **User:** Yes, I'd like to reserve a car there. I'd like to pick it up at Indira Gandhi International Airport at 3 o'clock in the afternoon. I'd like a Hatchback. **RealTOD:** I'll help you reserve a Hatchback at Indira Gandhi International Airport. Would you like to pick up the car on the same date at 3:00 PM? Also, how long will you keep the car? **User:** Yes, that sounds good. I'll return the car on the 3rd of this month. **RealTOD:** I'll help you reserve a Hatchback from March 2nd to March 3rd. Would you like to add insurance to your rental? **User:** Yes, I'd like to add insurance.
RealTOD: APICall(method='ReserveCar', parameters={ pickup_location: Indira Gandhi International Airport, car_type: Hatchback, start_date: 2019-03-02, end_date: 2019-03-03, pickup_time: 15:00, add_insurance: True }) ✓
Search Results: [{'car_name': 'Fiat Panda', 'car_type': 'Hatchback', 'pickup_location': 'Indira Gandhi International Airport', 'pickup_time': '15:00', 'price_per_day': 39.0, 'add_insurance': True}]
RealTOD: Your car reservation has been confirmed. You'll be getting a Fiat Panda Hatchback, which will cost $39.00 per day with insurance included. You can pick it up at Indira Gandhi International Airport on March 2nd at 3:00 PM. Would you like to know any additional details about your car rental? **User:** No, thank you for your help. **RealTOD:** I'm happy to assist. Have a great day!

Table 8: Multi-SGD Conversation - RealTOD (GPT-4o)

SGD Multi-Domain Conversation (GPT-4o)
Goal: User is trying to check the weather in Vancouver, then schedule a property visit at the Golf Club Manor Apartments, and reserve a Hatchback car
User: What's the weather going to be like in Vancouver, BC on March 2nd?
RealTOD: APICall(method='GetWeather', parameters={ city: Vancouver, date: 2019-03-02 }) ✓
Search Results: [{'city': 'Vancouver', 'date': '2019-03-02', 'temperature': 67, 'precipitation': 49, 'humidity': 45, 'wind': 15}]
RealTOD: The average temperature for the day will be about 67 degrees Fahrenheit, and there's a 49 percent chance of rain. **User:** That sounds good. I'd like to schedule a visit to Golf Club Manor Apartments. **RealTOD:** Could you confirm that you want to visit Golf Club Manor Apartments on March 2nd? **User:** Sounds good.
RealTOD: APICall(method='ScheduleVisit', parameters={ property_name: Golf Club Manor Apartments, visit_date: 2019-03-02 }) ✓
Search Results: [{'property_name': 'Golf Club Manor Apartments', 'address': '375 Industrial Parkway # 314', 'number_of_baths': 3, 'number_of_beds': 3, 'phone_number': '510-581-0911', 'price': 3950}]
RealTOD: Your visit has been scheduled. **User:** I would like to reserve a car there till the 3rd. **RealTOD:** Do you like Hatchback, SUV, or another type of car? Would you like to pick up the car from March 2nd and from where? **User:** Sounds good. Delhi Airport is a good place for me to pick it up. A hatchback sounds perfect. **RealTOD:** Would you like insurance? What time will you pick it up? **User:** I'd like insurance and will pick it up at 3:00 PM. **RealTOD:** So you want to pick up the car from Indira Gandhi International Airport on March 2nd at 3 PM and drop it off on March 3rd? It's a Hatchback that comes with insurance. **User:** Sounds perfect.
RealTOD: APICall(method='ReserveCar', parameters={ add_insurance: True, car_type: Hatchback, end_date: 2019-03-03, pickup_location: Indira Gandhi International Airport, pickup_time: 15:00, start_date: 2019-03-02 }) ✓
Search Results: [{'car_name': 'Hyundai i20', 'car_type': 'Hatchback', 'pickup_location': 'Indira Gandhi International Airport', 'pickup_time': '15:00', 'price_per_day': 35.00, 'add_insurance': True}]
RealTOD: Your reservation for a Hyundai i20 Hatchback has been confirmed. The cost is $35.00 per day. If you have any more questions or need further assistance, feel free to ask! **User:** Thank you. That's all I need for now. **RealTOD:** Have a great day!

Table 9: Multi-SGD Conversation - RealTOD (DeepSeek-V3)

SGD Multi-Domain Conversation (DeepSeek-V3)
Goal: User is trying to check the weather in Vancouver, then schedule a property visit at the Golf Club Manor Apartments, and reserve a Hatchback car
User: What's the weather going to be like in Vancouver, BC on March 2nd?
RealTOD: APICall(method='GetWeather', parameters={ city: Vancouver, date: 2019-03-02 }) ✓
Search Results: [{'city': 'Vancouver', 'date': '2019-03-02', 'temperature': 45, 'precipitation': 10, 'humidity': 65, 'wind': 12}]
RealTOD: On March 2nd in Vancouver, BC, the temperature will be 45 degrees Fahrenheit with a 10 percent chance of rain. The wind speed will be 12 mph. **User:** I'd like to schedule a visit to the Golf Club Manor Apartments. **RealTOD:** Got it! I can help you schedule a visit to the Golf Club Manor Apartments. Could you please confirm the date you'd like to schedule the visit for? **User:** I'd like to visit the property on the 2nd of this month.
RealTOD: APICall(method='ScheduleVisit', parameters={ property_name: Golf Club Manor Apartments, visit_date: 2019-03-02 }) ✓
Search Results: [{'property_name': 'Golf Club Manor Apartments', 'address': '123 Fairway Drive', 'number_of_baths': 2, 'number_of_beds': 3, 'phone_number': '604-555-1234', 'price': 2500}]
RealTOD: Your visit to the Golf Club Manor Apartments on March 2nd has been confirmed. The property is located at 123 Fairway Drive and has 3 bedrooms and 2 bathrooms. Let me know if you'd like any additional details! **User:** I'd like to book a car to get there. I'd like to pick it up at Indira Gandhi International Airport, and I'd like it to be a Hatchback. **RealTOD:** I'll help you book a Hatchback for your trip. Could you please provide the date and time you'd like to pick up the car, the return date, and whether you'd like to add insurance? **User:** I'd like to pick up the car at 3 o'clock in the afternoon, and I'd like to return it on the 3rd of this month. I'd like to add insurance to my rental.
RealTOD: APICall(method='ReserveCar', parameters={ car_type: Hatchback, pickup_location: Indira Gandhi International Airport, start_date: 2019-03-02, pickup_time: 15:00, end_date: 2019-03-03, add_insurance: True }) ✓
Search Results: [{'car_name': 'Fiat Panda', 'car_type': 'Hatchback', 'pickup_location': 'Indira Gandhi International Airport', 'pickup_time': '15:00', 'price_per_day': 39.0, 'add_insurance': True}]
RealTOD: Your car reservation has been successfully made! You'll be getting a Fiat Panda Hatchback, which will cost $39.00 per day with insurance included. You can pick it up at Indira Gandhi International Airport on March 2nd at 3:00 PM. Let me know if you'd like any additional details! **User:** Thank you for your assistance. That will be all for now. **RealTOD:** I'm happy to assist. Have a great day!

Table 10: Multi-SGD Conversation - RealTOD (Llama-3.3-70B-Instruct)

SGD Multi-Domain Conversation (Llama-3.3-70B-Instruct)
Goal: User is trying to check the weather in Vancouver, then schedule a property visit at the Golf Club Manor Apartments, and reserve a Hatchback car
User: What's the weather going to be like in Vancouver, BC on March 2nd? **RealTOD:** To provide you with the weather information for Vancouver, BC on March 2nd, I'll need to make an API call. Before I do that, I just want to confirm the details with you. You're looking for the weather in Vancouver, BC, and the specific date you're interested in is March 2nd, correct? **User:** Yes, that's correct.
RealTOD: APICall(method='GetWeather', parameters={ city: Vancouver, date: 2019-03-02 }) ✓
Search Results: [{'city': 'Vancouver', 'date': '2019-03-02', 'temperature': 68, 'precipitation': 25, 'humidity': 26, 'wind': 6}]
RealTOD: It seems like the search results provided multiple weather forecasts for Vancouver, BC on March 2nd. To provide a clear answer, could you specify if you're more interested in temperature, precipitation, or humidity? **User:** Yes, that's what I'm looking for. I'd like to schedule a visit to the Golf Club Manor Apartments on that day. **RealTOD:** To schedule a visit to the Golf Club Manor Apartments, I need to confirm the details with you. You'd like to visit on March 2nd, correct? **User:** Yes, that's correct.
RealTOD: APICall(method='ScheduleVisit', parameters={ property_name: Golf Club Manor Apartments, visit_date: 2019-03-02 }) ✓
Search Results: [{'property_name': 'Golf Club Manor Apartments', 'address': '375 Industrial Parkway # 314', 'number_of_baths': 1, 'number_of_beds': 2, 'phone_number': '5105810911', 'price': 2500000}]
RealTOD: Your visit to Golf Club Manor Apartments has been scheduled. The property is located at 375 Industrial Parkway # 314. **User:** I'd like to reserve a car there and pick it up at Indira Gandhi International Airport. **RealTOD:** Would you like a Sedan, Hatchback, or SUV? Also, what time would you like to pick it up? **User:** I'd like a Hatchback and will pick it up at 3:00 PM.
RealTOD: APICall(method='ReserveCar', parameters={ add_insurance: True, car_type: Hatchback, end_date: 2019-03-03, pickup_location: Indira Gandhi International Airport, pickup_time: 15:00, start_date: 2019-03-02 }) ✓
Search Results: [{'car_name': 'Fiat Panda', 'car_type': 'Hatchback', 'pickup_location': 'Indira Gandhi International Airport', 'pickup_time': '15:00', 'price_per_day': 39.00, 'add_insurance': True}]
RealTOD: Your reservation for a Fiat Panda Hatchback has been confirmed. The cost is $39.00 per day. **User:** Thank you very much. You've been very helpful. That will be all for now. **RealTOD:** Have a great day!

Table 11: Multi-SGD Conversation Baseline Model Dialog Example -(SOLOIST)

SGD Multi-Domain Conversation (SOLOIST)
Goal: User is trying to check the weather in Vancouver, then schedule a property visit at the Golf Club Manor Apartments, and reserve a Hatchback car
User: I'd like to find out what the weather is going to be like on 2nd of March in Vancouver, BC.
SOLOIST: APICall: (Missed API Call Here) ✗
Search Results: [Missed Search Results] ✗
SOLOIST: The average temperature for the day should be 67 degrees Fahrenheit with a 49 percent chance of rain. **User:** That sounds good. I'd like to schedule a visit to Golf Club Manor Apartments. **SOLOIST:** What time do you want to pick it up? **User:** Sounds good.
SOLOIST: APICall(method='ScheduleVisit', parameters={ property_name: Golf Club Manor Apartments, visit_date: 2019-03-02nd }) ✓
Search Results: [{'address': '375 Industrial Parkway # 314', 'has_garage': False, 'in_unit_laundry': True, 'number_of_baths': 3, 'number_of_beds': 3, 'phone_number': '510-581-0911', 'price': 3950, 'property_name': 'Golf Club Manor Apartments', 'visit_date': '2019-03-02'}]
SOLOIST: Your visit has been scheduled. **User:** I would like to reserve a car there till the 3rd. **SOLOIST:** Could I help you with anything else? **User:** Sounds good. Delhi Airport is a good place for me to pick it up. A hatchback sounds perfect. **SOLOIST:** Please confirm: You want me to reserve a standard car from YVR International Airport on March 2nd at 3:30 pm until March 11th. **User:** I'd like insurance and will pick it up at 3 PM. **SOLOIST:** Do you want to pick it up on March 1st? You'll be picking it up at 3:30 pm. Is that right? **User:** Sounds perfect.
SOLOIST: APICall(method='GetCarsAvailable', parameters={ add_insurance: True, car_type: Standard, end_date: 2019-03-03, pickup_location: YVR International Airport, pickup_time: 15:30, start_date: 2019-03-01 }) ✗
Search Results: [{'car_name': 'Hyundai Elantra', 'car_type': 'Standard', 'pickup_location': 'VVR International Airport', 'pickup_time': '15:30', 'price_per_day': 34.0, 'add_insurance': True}]
SOLOIST: Your car has been reserved. **SOLOIST:** The car has been reserved. It is a Hyundai Elantra and it is $34. **User:** What is the name of the car and how much is it per day? **SOLOIST:** It is $34.00 per day and it's called a Hyundai Elantra. **User:** Thanks for helping. That's it for now. **SOLOIST:** Have a great day.

Table 12: Multi-SGD Conversation Baseline Model Dialog Example - (ZS-TOD)

SGD Multi-Domain Conversation (ZS-TOD)
Goal: User is trying to check the weather in Vancouver, then schedule a property visit at the Golf Club Manor Apartments, and reserve a Hatchback car
User: I'd like to find out what the weather is going to be like on 2nd of March in Vancouver, BC.
ZS-TOD: APICall(method='GetWeather', parameters={ city: Vancouver, date: 2019-03-02 }) ✓
Search Results: [city: Vancouver, date: 2019-03-02, humidity: 45, precipitation: 49, temperature: 67, wind: 15]
ZS-TOD: The average temperature for the day will be about 67 degrees Fahrenheit and there's a 49 percent chance of rain. **User:** That sounds good. I'd like to schedule a visit to Golf Club Manor Apartments. **ZS-TOD:** Could you confirm that you want to visit Golf Club Manor Apartments on March 2nd? **User:** Sounds good.
ZS-TOD: APICall(method='ScheduleVisit', parameters={ property_name: Golf Club Manor Apartments, visit_date: 2019-03-02 }) ✓
Search Results: [address: 375 Industrial Parkway # 314, has_garage: False, in_unit_laundry: True, number_of_baths: 3, number_of_beds: 3, phone_number: 510-581-0911, price: 3950, property_name: Golf Club Manor Apartments, visit_date: 2019-03-02]
ZS-TOD: Your visit has been scheduled. **User:** I would like to reserve a car there till the 3rd. **ZS-TOD:** Do you like Hatchback, SUV, or another type of car? Would you like to pick up the car from March 2nd and from where? **User:** Sounds good. Delhi Airport is a good place for me to pick it up. A hatchback sounds perfect. **ZS-TOD:** Would you like insurance? What time will you pick it up? **User:** I'd like insurance and will pick it up at 3 PM. **ZS-TOD:** So you want to pick up the car from Indira Gandhi International Airport on March 2nd at 3 PM and drop it off on March 3rd? It's a Hatchback that comes with insurance.
ZS-TOD: APICall: (Missed API Call Here) ✗
Search Results: [Missed Search Results] ✗
User: What is the name of the car and how much is it per day? **ZS-TOD:** It is $39.00 per day and it's called a Fiat Panda. **User:** Thanks for helping. That's it for now. **ZS-TOD:** Have a great day.

E User Study Instructions

Disclaimers of any risks to participants or annotators

There are no significant risks associated with participating in this study. However, annotators may experience mild fatigue or cognitive strain due to prolonged reading and evaluation of multiple conversations. If you feel discomfort or fatigue, please take breaks as needed.

Instructions for Human Study Participants

Your task is to evaluate model-generated responses in multi-turn, task-oriented conversations based on the following criteria:

1. Fluency
2. Informativeness
3. Task Completion

Task Overview

- You will be presented with multiple conversations, where a user interacts with a model to complete a task (e.g., booking a flight).
- Your job is to rate each model response independently using a 1-5 scale based on the provided criteria.
- This study is blind — you will not know which model produced which response.

How to Rate Responses

You will assign a score for each response based on the following:

1. Fluency

Measures whether the response is grammatically correct, well-structured, and natural to read.

- 1 = Unnatural, grammatically incorrect, or hard to read.
- 5 = Perfectly fluent, natural, and error-free.

Example (Good Fluency)

User: I need to book a flight to New York next Monday.
Model: Sure! What time would you like to depart?

Example (Poor Fluency)

User: I need to book a flight to New York next Monday.
Model: Sure! You next Monday flight want time what?

2. Informativeness

Measures whether the response provides useful and relevant information to advance the task.

- 1 = Vague, incorrect, or unhelpful.
- 5 = Precise, relevant, and useful.

Example (Good Informativeness)

User: Can you recommend a vegetarian restaurant nearby?
Model: Yes! Green Leaf Café is a highly rated vegetarian restaurant.

Example (Low Informativeness)

User: Can you recommend a vegetarian restaurant nearby?
Model: There are many restaurants in your area.

3. Task Completion

Measures whether the response successfully progresses or completes the task in the conversation.

- 1 = Fails to address the request.
- 5 = Fully completes the task.

Example (Good Task Completion)

User: I'd like to book a table for two at an Italian restaurant at 7 PM today.
Model: I found a reservation at Bella Italia for 7 PM. Would you like me to reserve it for 2 people at 7 PM today?

Example (Failed Task Completion)

User: I'd like to book a table for two at an Italian restaurant at 7 PM.
Model: There are Italian restaurants in your area.

Additional Guidelines

- Be objective — rate based on quality, not personal preference.
- If a response is unclear or ambiguous, leave a comment.
- Do not try to guess which model produced the response.

Study Duration & Submission

- The study will take approximately 10 minutes to complete.
- Once you have evaluated all responses, submit your ratings.